

Yes — this is **very important** because "public/private inheritance" sounds confusing until you see what it actually controls.

Public vs Private Inheritance

Suppose we have:

```
class Animal {
public:
    void eat() {}

protected:
    int age;

private:
    int salary;
};
```

And:

```
class Dog : public Animal {
};
```

The `public` here means **how the inherited members' access levels are exposed through `Dog`**.

It does **not** mean whether `Dog` gets the members or not.

1. Public inheritance

```
class Dog : public Animal {
};
```

Think:

"Dog IS-A Animal."

The access levels from `Animal` are preserved:

`Animal` memberIn `Dog``publicpublicprotectedprotectedprivate` inaccessible directly

Example:

```
class Animal {
public:
    void eat() {}
protected:
    int age;
private:
    int salary;
};

class Dog : public Animal {
public:
    void test() {
        eat();           // ☑
        age = 5;         // ☑
        // salary = 10;  // ✗
    }
};
```

And outside:

```
Dog d;

d.eat();    // ☑
d.age = 5;  // ✗ protected
```

So **public inheritance preserves the accessibility**.

2. Private inheritance

Now:

```
class Dog : private Animal {  
};
```

This means:

"Dog inherits Animal's implementation, but Animal's public/protected interface is NOT exposed publicly through Dog."

The access levels become:

`Animal` memberIn `Dogpublicprivateprotectedprivateprivate` inaccessible

Example:

```
class Animal {  
public:  
    void eat() {}  
protected:  
    int age;  
};  
  
class Dog : private Animal {  
public:  
    void test() {  
        eat();      // ☒  
        age = 5;    // ☒  
    }  
};
```

But:

```
Dog d;  
  
d.eat();    // ☒
```

Why?

Because `eat()` was `public` in `Animal`, but **private inheritance converted it to private inside `Dog`**.

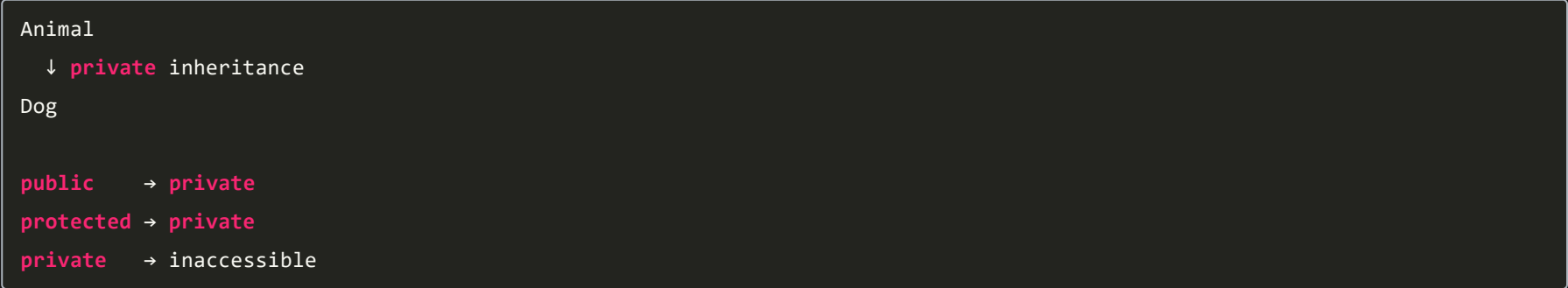
3. Think of inheritance as a visibility filter

This mental model is extremely useful:

Public inheritance



Private inheritance



4. What about `protected`?

This is where the three access modifiers connect.

Suppose:

```
class Animal {
public:
    int a;

protected:
    int b;

private:
    int c;
};
```

With:

```
class Dog : public Animal
```

Inside **Dog**:

```
a; // ☒ public
b; // ☒ protected
c; // ☐ inaccessible
```

Outside **Dog**:

```
Dog d;

d.a; // ☒
d.b; // ☐
d.c; // ☐
```

5. Why would we use private inheritance?

It's much less common than public inheritance.

Public inheritance expresses an **IS-A** relationship:

```
Dog IS-A Animal
Car IS-A Vehicle
Circle IS-A Shape
```

So normally:

```
class Dog : public Animal
```

Private inheritance is more about:

```
"I want to reuse the implementation of this class, but I don't want users of my class to treat me as that base class."
```

For most modern C++ designs, **composition is often preferred over private inheritance** for this kind of reuse.

6. The BIG distinction

Don't confuse:

```
class Dog : public Animal
```

with:

```
class Dog {
public:
    Animal animal;
};
```

These are different concepts.

Inheritance

```
class Dog : public Animal
```

means:

```
Dog is an Animal.
```

Composition

```
class Dog {
    Animal animal;
};
```

means:

```
Dog has an Animal.
```

This **IS-A vs HAS-A** distinction is extremely important in OOP interviews.

7. Why did **struct** vs **class** matter?

Now the previous topic should make sense.

When you write:

```
struct Dog : Animal {  
};
```

you are implicitly writing:

```
struct Dog : public Animal {  
};
```

But:

```
class Dog : Animal {  
};
```

means:

```
class Dog : private Animal {  
};
```

because their **default inheritance access differs**.

Memorize this:

```
struct:  
  members → public  
  inheritance → public  
  
class:  
  members → private  
  inheritance → private
```

One interview question you should now be able to answer:

```
What happens to public and protected members under public vs private inheritance?
```

Public inheritance: public stays public, protected stays protected.

Private inheritance: public and protected become private.

Private base members remain inaccessible in both cases.